

Contents

Final target architecture	3
Phase 1 — Verify base system first	3
1.1 Host checks	4
1.2 Docker checks	4
1.3 Kubernetes checks, if your RIC/SMO uses Kubernetes	4
1.4 Port checks	5
Phase 2 — Verify Open5GS	5
2.1 Check Open5GS services	5
2.2 Check SCTP for AMF	6
2.3 Check AMF config values	6
2.4 Check UPF tunnel	6
2.5 Check subscriber	7
Phase 3 — Verify USRP B210 path, because ZMQ is not installed	7
3.1 Check B210 detection	7
3.2 Check OCUDU gNB config	8
3.3 Start Open5GS first	8
3.4 Start OCUDU gNB/CU/DU	8
3.5 Start UE traffic if available	9
Phase 4 — Verify Near-RT RIC separately	9
Phase 5 — Verify SMO / ONAP / Non-RT RIC readiness	10
5.1 Check SMO / SDN-R	10
5.2 Check VES collector	10
5.3 Check Non-RT RIC RAN PM pods	11
Phase 6 — Install and verify OCUDU NETCONF	11
6.1 Build NETCONF container	12
6.2 Create Docker network	12
6.3 Run NETCONF service	12
6.4 Validate NETCONF manually	13
Phase 7 — Install and verify OCUDU O1 adapter	13
7.1 Clone and build	13
7.2 Start O1 adapter in basic CM/FM mode	13
7.3 Test config-change detection	14
Phase 8 — Connect O1 to SMO / SDN-R	15
Phase 9 — Understand PM data options before implementing	15
Option 1 — OCUDU WebSocket PM, easiest practical path	15
Option 2 — Standard O-RAN file-based PM	16

Phase 10 — Install/check RAN PM components in Non-RT RIC	16
Phase 11 — Verify RAN PM pipeline without OCUDU first	18
Phase 12 — Add OCUDU PM data path	18
12.1 Check OCUDU WebSocket port	19
12.2 Start O1 adapter with WebSocket and VES parameters	19
Phase 13 — Build minimal OCUDU PM exporter	20
13.1 First exporter goal	20
13.2 Add Kafka output	21
Phase 14 — Convert OCUDU PM to rApp training format	23
Phase 15 — Standard O1 PM fileReady path	23
15.1 Required standard O-RAN PM objects	24
15.2 PM file-ready flow to test	24
15.3 Minimal PM XML strategy	24
15.4 Send VES fileReady event	25
Phase 16 — Verify PM data reaches rApp	26
16.1 Check VES collector	26
16.2 Check DFC	26
16.3 Check MinIO	26
16.4 Check PM Producer	27
16.5 Check PM rApp	27
Phase 17 — Recommended order for you	27
What you should not do yet	28
Final recommendation	28

Yes. But first, one important correction:

With OCUDU today, O1 adapter does not provide full standard O1 PM by itself. OCUDU documentation says the O1 adapter currently supports **CM** and **FM**, while **PM is implemented through OCUDU's JSON WebSocket metrics service.** (ocudu-docs-604e90.gitlab.io)

So for your goal — **O1 with PM data for rApp training** — you need one of these two paths:

Path A: Practical OCUDU path

OCUDU WebSocket PM metrics → O1/PM exporter → VES/Kafka/InfluxDB/Non-RT RIC → rApp

Path B: Standard O-RAN PM path

RAN node generates 3GPP PM XML file → VES fileReady event → PM Data File Collector → PM File

The standard O-RAN-SC RAN PM flow expects a RAN node to send a **VES fileReady event**, then the Data File Collector fetches the PM XML file, stores it, the PM File Converter converts it to JSON, and the PM Producer distributes filtered PM data to rApps over Kafka. (docs.o-ran-sc.org)

Because you said **ZMQ is not installed**, the first verification should use your current **USRP B210** path, or run the O1/PM pipeline without UE traffic first. Without ZMQ or real UE traffic, PM data will be limited.

Final target architecture

Use this as your target:

Open5GS Core

N2/N3

OCUDU gNB / CU / DU

B210 RF path, no ZMQ

E2 → Near-RT RIC

O1 NETCONF → SMO / SDN-R

O1 FM/VES → VES Collector → Kafka

PM metrics WebSocket → PM exporter → Kafka / InfluxDB / rApp

Non-RT RIC

ICS / PM Producer

InfluxDB

Kafka

rApp training pipeline

Phase 1 — Verify base system first

Do not install O1 PM until these checks pass.

1.1 Host checks

```
hostname -I
date
timedatectl
free -h
df -h
lscpu | egrep 'Model name|CPU\(s\)'
```

Expected:

```
Correct time
Enough RAM
Enough disk
Stable host IP
```

For full ONAP/SMO/Non-RT RIC + RAN PM, you need a strong machine. If ONAP, Near-RT RIC, Non-RT RIC, Open5GS, and OCUDU are all on one host, I recommend at least:

```
CPU: 12+ cores
RAM: 64 GB preferred
Disk: 200 GB+
```

1.2 Docker checks

You prefer `sudo docker`, so use:

```
sudo docker --version
sudo docker compose version
sudo docker ps
sudo docker network ls
```

Check for unhealthy containers:

```
sudo docker ps --filter "health=unhealthy"
```

Check disk usage:

```
sudo docker system df
```

Do not prune yet unless you know which images/volumes are unused.

1.3 Kubernetes checks, if your RIC/SMO uses Kubernetes

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get svc -A
```

Check bad pods:

```
kubectl get pods -A | egrep -i 'crash|error|pending|imagepull|terminating'
```

Expected:

All required pods Running or Completed

No CrashLoopBackOff

No ImagePullBackOff

No Pending pods

1.4 Port checks

```
sudo ss -tulpen | egrep '(:830|:5000|:8001|:8181|:8080|:8443|:38412|:2152)'
```

Important ports:

Port	Meaning
830/tcp	O1 NETCONF
5000/tcp	OCUDU O1 adapter REST API
8001/tcp	OCUDU WebSocket metrics/control
8181/tcp	SDN-R / OpenDaylight / ODLUX, depending on deployment
8080/tcp	VES HTTP
8443/tcp	VES HTTPS
38412/sctp	NGAP AMF
2152/udp	GTP-U

Phase 2 — Verify Open5GS

2.1 Check Open5GS services

If installed as services:

```
systemctl list-units 'open5gs*' --type=service --all
```

```
sudo systemctl status open5gs-amfd --no-pager
sudo systemctl status open5gs-smfd --no-pager
sudo systemctl status open5gs-upfd --no-pager
sudo systemctl status open5gs-nrfd --no-pager
sudo systemctl status open5gs-ausfd --no-pager
sudo systemctl status open5gs-udmd --no-pager
sudo systemctl status open5gs-udrd --no-pager
```

If Docker-based:

```
sudo docker ps | grep -i open5gs
sudo docker logs --tail=100 <open5gs-amf-container>
sudo docker logs --tail=100 <open5gs-upf-container>
```

Expected:

```
AMF running
SMF running
UPF running
AUSF/UDM/UDR/NRF running
MongoDB running if used
```

2.2 Check SCTP for AMF

```
lsmod | grep sctp || true
sudo modprobe sctp
sudo ss -lnp --sctp
```

Expected:

AMF listening on 38412/SCTP

If nothing appears:

```
sudo grep -R "ngap" -n /etc/open5gs/amf.yaml
sudo journalctl -u open5gs-amfd --no-pager -n 100
```

2.3 Check AMF config values

```
sudo cat /etc/open5gs/amf.yaml | sed -n '1,220p'
```

Write down:

```
AMF NGAP IP
MCC
MNC
TAC
SST
SD, if used
```

Your OCUDU gNB config must match these.

2.4 Check UPF tunnel

```
ip addr show ogstun || true
ip route | grep ogstun || true
```

```
sudo iptables -t nat -S | grep -E '10.45|MASQUERADE' || true
```

Typical Open5GS UE subnet:

10.45.0.0/16

If UE needs internet access:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
sudo iptables -t nat -A POSTROUTING \
-s 10.45.0.0/16 ! -o ogstun -j MASQUERADE
```

2.5 Check subscriber

Your UE/test SIM must be in Open5GS.

Check:

IMSI

K

OPc or OP

AMF value

MCC/MNC

DNN/APN

SST/SD

Without a valid subscriber, UE attach will fail, and you will not get meaningful PM traffic.

Phase 3 — Verify USRP B210 path, because ZMQ is not installed

Since ZMQ is not installed, use your B210 path first.

3.1 Check B210 detection

```
lsusb | grep -i ettus
```

```
sudo uhd_find_devices
```

```
sudo uhd_usrp_probe
```

Check USB speed:

```
lsusb -t
```

Expected:

```
USRP B210 visible
USB speed 5000M
uhd_usrp_probe succeeds
```

If B210 is on USB2, fix cabling/port before continuing.

3.2 Check OCUDU gNB config

Find your config:

```
find ~ -type f \( -name '*gnb*.yaml' -o -name '*gnb*.yml' -o -name '*du*.yaml' -o -name '*du*.yml' \)
```

Check these values:

```
cu_cp:
  amf:
    addr: <OPEN5GS_AMF_IP>
    port: 38412
    bind_addr: <GNB_LOCAL_IP>
```

```
cell_cfg:
  plmn: "<MCCMNC>"
  tac: <TAC>
```

Check B210 section:

```
ru_sdr:
  device_driver: uhd
  device_args: type=b200
  srates: 23.04
  tx_gain: <your-value>
  rx_gain: <your-value>
```

3.3 Start Open5GS first

```
sudo systemctl restart open5gs-amfd open5gs-smfd open5gs-upfd
sudo journalctl -u open5gs-amfd -f
```

Leave AMF logs open in one terminal.

3.4 Start OCUDU gNB/CU/DU

In another terminal, start your OCUDU gNB/CU/DU exactly as you currently do.

Expected gNB side:

AMF connection established
NG Setup Request sent
NG Setup Response received
Cell started

Expected AMF side:

gNB connected
NG setup

If gNB cannot connect to AMF, stop here. O1 PM will not be useful until this works.

3.5 Start UE traffic if available

With B210, you need:

COTS UE + test SIM
or SDR UE
or Amarisoft UE simulator

Expected Open5GS logs:

Registration request
Authentication success
Security mode complete
Initial context setup
PDU session establishment
UE IP assigned

If you do not have UE traffic, you can still verify O1 CM/FM and PM pipeline plumbing, but PM counters will be weak or synthetic.

Phase 4 — Verify Near-RT RIC separately

O1 is independent of E2, but your whole system should be checked.

```
kubectl get pods -A | grep -Ei 'ric|e2|submgr|appmgr|rtmgr|e2mgr' || true
```

If Docker-based:

```
sudo docker ps | grep -Ei 'ric|e2|submgr|appmgr|rtmgr'
```

Check E2 endpoint:

```
nc -vz <RIC_E2TERM_IP> 36421
```

Expected:

E2Term reachable
gNB appears as E2 node
xApps can subscribe later

Do not debug PM and E2 at the same time. First make O1/PM data flow work.

Phase 5 — Verify SMO / ONAP / Non-RT RIC readiness

You need three separate functions:

1. SDN-R / OpenDaylight for O1 NETCONF
2. VES Collector for O1 FM/PM events
3. Non-RT RIC RAN PM pipeline for PM files and rApp subscription

O-RAN-SC SMO VES contains a VES collector, Kafka bus, InfluxDB connector, DMaAP adapter, and Grafana dashboard. (docs.o-ran-sc.org)

5.1 Check SMO / SDN-R

Try browser:

`http://<SMO_HOST_IP>:8181/odlux/index.html`

Check pods/containers:

```
kubectl get pods -A | grep -Ei 'sdnr|odl|odlux|controller|ves|kafka|influx|minio|nonrtric'
```

or Docker:

```
sudo docker ps | grep -Ei 'sdnr|odl|odlux|controller|ves|kafka|influx|minio|nonrtric'
```

Expected:

SDN-R / ODL running

VES Collector running

Kafka running

InfluxDB running, if used

MinIO running, if RAN PM file pipeline is used

5.2 Check VES collector

If Kubernetes:

```
kubectl get svc -A | grep -Ei 'ves|collector'
```

```
kubectl get pods -A | grep -Ei 'ves|collector'
```

Check reachability:

```
nc -vz <VES_HOST_IP> 8080
nc -vz <VES_HOST_IP> 8443
```

For O-RAN-SC RANPM-only deployment examples, VES Collector node ports are listed as 31760 HTTP and 31761 HTTPS. (lf-o-ran-sc.atlassian.net)

So also check:

```
nc -vz <K8S_NODE_IP> 31760
nc -vz <K8S_NODE_IP> 31761
```

5.3 Check Non-RT RIC RAN PM pods

If you already installed RANPM:

```
kubectl get pods -n nonrtric
```

Expected components include things like:

```
dfc
ves-collector
message-router
kafka
minio
influxdb2
informationservice
pm-producer
pmlog
pm-rapp
```

O-RAN-SC's RANPM-only example includes pods such as `dfc`, `influxdb2`, `informationservice`, `kafka`, `message-router`, `minio`, `pm-producer`, `pmlog`, `pm-rapp`, and `ves-collector`. (lf-o-ran-sc.atlassian.net)

Check services:

```
kubectl get svc -n nonrtric
```

Check bad pods:

```
kubectl get pods -n nonrtric | egrep -i 'crash|error|pending|imagepull'
```

If any PM pod is not running, fix that before connecting OCUDU PM.

Phase 6 — Install and verify OCUDU NETCONF

The NETCONF service must come before the O1 adapter.

6.1 Build NETCONF container

```
cd ~
```

```
git clone https://gitlab.com/ocudu/ocudu_elements/ocudu_oran_apps/ocudu_netconf.git
cd ocudu_netconf
```

```
sudo docker build -t ocudu-netconf/ocudu-netconf:latest . --progress=plain
```

OCUDU documents this Docker build command for the NETCONF service.
(ocudu-docs-604e90.gitlab.io)

6.2 Create Docker network

```
sudo docker network create smo_integration 2>/dev/null || true
sudo docker network ls | grep smo_integration
```

If your SMO/ONAP is already on another Docker network, later connect `ocudu-netconf` to that network too.

6.3 Run NETCONF service

Use the documented default-config mode first:

```
sudo docker rm -f ocudu-netconf 2>/dev/null || true
```

```
sudo docker run -d \
  --name ocudu-netconf \
  --network smo_integration \
  -p 830:830 \
  -v "$(pwd)/default_config.xml:/config/default_config.xml" \
  ocudu-netconf/ocudu-netconf:latest \
  /config/default_config.xml
```

OCUDU documents running the standalone NETCONF server with host port 830 and a mounted `default_config.xml`. (ocudu-docs-604e90.gitlab.io)

Check logs:

```
sudo docker logs -f ocudu-netconf
```

Check port:

```
sudo ss -tulpen | grep ':830'
```

6.4 Validate NETCONF manually

```
sudo docker exec -it ocudu-netconf bash
```

Inside:

```
netopeer2-cli
```

Then:

```
connect --login root
get-config --source=running
```

OCUDU documents testing the service with `netopeer2-cli`, `connect --login root`, and `get-config --source=running`. (ocudu-docs-604e90.gitlab.io)

Expected:

```
NETCONF session opens
running datastore is returned
```

Phase 7 — Install and verify OCUDU O1 adapter

7.1 Clone and build

```
cd ~
```

```
git clone https://gitlab.com/ocudu/ocudu_elements/ocudu_oran_apps/ocudu_o1_adapter.git
cd ocudu_o1_adapter
```

```
sudo docker build -t ocudu-o1-adapter:latest .
```

7.2 Start O1 adapter in basic CM/FM mode

```
sudo mkdir -p /opt/ocudu-o1
sudo chmod 777 /opt/ocudu-o1
```

```
sudo docker rm -f ocudu-o1-adapter 2>/dev/null || true
```

```
sudo docker run -d \
  --name ocudu-o1-adapter \
  --network smo_integration \
  -p 5000:5000 \
  -v /opt/ocudu-o1:/shared \
  ocudu-o1-adapter:latest \
  python3 ./src/o1_adapter.py \
```

```
--netconf_host ocudu-netconf \
--netconf_port 830 \
--netconf_username root \
--netconf_password root \
--config /shared/ocudu-generated-gnb.yaml \
--template gnb.yaml \
--loglevel INFO
```

Check:

```
sudo docker logs -f ocudu-o1-adapter
curl -i http://localhost:5000/config-healthy
ls -l /opt/ocudu-o1
```

OCUDU says the adapter connects to the NETCONF datastore at startup, generates an initial config from the `running` datastore, and exposes `/config-healthy` on port 5000. (ocudu-docs-604e90.gitlab.io)

Expected:

```
O1 adapter connected to NETCONF
Generated YAML file exists
/config-healthy returns HTTP 200
```

7.3 Test config-change detection

Modify NETCONF running datastore:

```
sudo docker exec -it ocudu-netconf bash
sysrepocfg -E nano --datastore running --format xml
```

OCUDU documents modifying the datastore with `sysrepocfg -E nano --datastore running --format xml`. (ocudu-docs-604e90.gitlab.io)

Then check:

```
curl -i http://localhost:5000/config-healthy
```

Expected after config modification:

```
HTTP 400
config unhealthy
```

Reset:

```
curl -H 'Content-Type: application/json' \
-d '{ "restarted": true }' \
-X POST http://localhost:5000/restarted
```

OCUDU documents the `/restarted` endpoint to reset the health state after restart. (ocudu-docs-604e90.gitlab.io)

Phase 8 — Connect O1 to SMO / SDN-R

Get NETCONF container IP:

```
sudo docker network inspect smo_integration | grep -i ipaddress
```

OCUDU explicitly says to inspect the `smo_integration` network to get the NETCONF container IP before adding OCUDU gNB/CU/DU as O-RAN components into SMO. (ocudu-docs-604e90.gitlab.io)

In SDN-R / ODLUX, add NETCONF node:

```
Node name: ocudu-gnb-o1
Host:      <ocudu-netconf-ip or host-ip>
Port:      830
Username:   root
Password:   root
Protocol:   NETCONF over SSH
```

Verify:

```
Node mounted
Capabilities visible
running datastore readable
```

If SMO cannot reach it:

```
sudo docker network connect <smo-docker-network> ocudu-netconf
```

Then test from SMO/controller container:

```
sudo docker exec -it <smo-controller-container> bash
nc -vz <ocudu-netconf-ip> 830
```

Phase 9 — Understand PM data options before implementing

You asked specifically for **PM data in O1**. There are two meanings.

Option 1 — OCUDU WebSocket PM, easiest practical path

OCUDU says PM is currently through a JSON metrics WebSocket service, not standard O1 PM XML. (ocudu-docs-604e90.gitlab.io)

So you collect:

OCUDU WebSocket metrics
↓
custom PM exporter
↓
VES measurement/perf3gpp or Kafka JSON
↓
InfluxDB / Non-RT RIC / rApp
This is the best first implementation.

Option 2 — Standard O-RAN file-based PM

This is the formal O-RAN-SC RANPM flow:

RAN node creates 3GPP PM XML file
↓
RAN node sends VES fileReady event
↓
VES Collector puts event on Kafka
↓
Data File Collector fetches PM XML file
↓
MinIO stores file
↓
PM File Converter converts XML → JSON
↓
PM Producer filters and distributes to rApps
↓
Influx Logger stores PM time series

O-RAN-SC documents PM reports as XML files based on 3GPP TS 32.432/32.435, collected from the RAN, converted to JSON, distributed by PM Producer, and optionally stored in InfluxDB. (docs.o-ran-sc.org)

This is better for a standards-based rApp, but you need either:

A RAN node that already generates standard PM XML + fileReady VES
or
a custom exporter that converts OCUDU metrics into standard PM XML + fileReady VES
or
an O-RU/O-DU 01 simulator that already generates fileReady events

Phase 10 — Install/check RAN PM components in Non-RT RIC

If your Non-RT RIC RANPM is not installed, install that first.

O-RAN-SC RANPM-only deployment uses scripts such as:

```
install-nrt.sh
install-pm-log.sh
install-pm-influx-job.sh
install-pm-rapp.sh
```

The documented RANPM-only installation says `install-nrt.sh` installs the main RANPM setup, `install-pm-log.sh` installs the producer for InfluxDB, `install-pm-influx-job.sh` sets up an alternative job for producing data stored in InfluxDB, and `install-pm-rapp.sh` installs an rApp that subscribes and prints received data. (lf-o-ran-sc.atlassian.net)

After install:

```
kubectl get pods -n nonrtric
kubectl get svc -n nonrtric
```

Expected:

```
dfc
ves-collector
message-router or kafka
minio
influxdb2
informationservice
pm-producer
pmlog
pm-rapp
```

Check exposed API ports. O-RAN-SC's RANPM example lists:

```
ICS:           31823 / 31824
VES Collector: 31760 / 31761
Redpanda UI:   31767
MinIO UI:      31768
InfluxDB UI:   31812
```

(lf-o-ran-sc.atlassian.net)

So test:

```
nc -vz <K8S_NODE_IP> 31760
nc -vz <K8S_NODE_IP> 31761
nc -vz <K8S_NODE_IP> 31823
nc -vz <K8S_NODE_IP> 31812
```

Phase 11 — Verify RAN PM pipeline without OCUDU first

Before connecting OCUDU, prove the RANPM pipeline works using test PM files or the included HTTPS server, if available.

Check PM pods:

```
kubectl logs -n nonrtric -l app=ves-collector --tail=100
kubectl logs -n nonrtric -l app=dfc --tail=100
kubectl logs -n nonrtric -l app=pm-producer --tail=100
kubectl logs -n nonrtric -l app=pmlog --tail=100
kubectl logs -n nonrtric -l app=pm-rapp --tail=100
```

If labels do not match:

```
kubectl get pods -n nonrtric
kubectl logs -n nonrtric <pod-name> --tail=100
```

Expected flow:

```
VES collector receives fileReady event
DFC receives fileReady event from Kafka
DFC downloads PM XML file
DFC stores file in MinIO / persistent volume
PM converter converts XML to JSON
PM Producer receives JSON file event
PM rApp receives filtered PM data
Influx logger stores selected PM data
```

Data File Collector specifically expects a **File Ready VES event**, fetches files from the RAN using SFTP/FTPES/HTTP/HTTPS, stores them in S3/file system, and emits a File Publish message to Kafka. (docs.o-ran-sc.org)

Phase 12 — Add OCUDU PM data path

Because OCUDU PM is WebSocket JSON, use this first:

```
OCUDU WebSocket :8001
↓
custom ocudu-pm-exporter
↓
Kafka topic: ocudu.pm.raw
↓
InfluxDB bucket/table
↓
rApp training
```

Then later convert to formal 3GPP XML/fileReady if needed.

12.1 Check OCUDU WebSocket port

On the gNB host:

```
sudo ss -tulpen | grep ':8001'
```

From O1 adapter host/container:

```
nc -vz <OCUDU_GNB_IP> 8001
```

If OCUDU runs in Docker:

```
sudo docker ps | grep -Ei 'ocudu|gnb|du|cu'
sudo docker inspect <ocudu-gnb-container> | grep -i IPAddress
```

12.2 Start O1 adapter with WebSocket and VES parameters

```
sudo docker rm -f ocudu-o1-adapter 2>/dev/null || true
```

```
sudo docker run -d \
  --name ocudu-o1-adapter \
  --network smo_integration \
  -p 5000:5000 \
  -v /opt/ocudu-o1:/shared \
  ocudu-o1-adapter:latest \
  python3 ./src/o1_adapter.py \
  --netconf_host ocudu-netconf \
  --netconf_port 830 \
  --netconf_username root \
  --netconf_password root \
  --config /shared/ocudu-generated-gnb.yaml \
  --template gnb.yaml \
  --ws_host <OCUDU_GNB_IP> \
  --ws_port 8001 \
  --ves_host <VES_HOST_OR_NODE_IP> \
  --ves_port 31760 \
  --ves_username sample1 \
  --ves_password sample1 \
  --loglevel INFO
```

Use 31760 if you are using the O-RAN-SC RANPM node port from the documented example. Use 8080 or 8443 if your VES collector exposes those directly.

Check:

```
sudo docker logs -f ocudu-o1-adapter
```

Expected:

```
Connected to NETCONF
Connected to WebSocket or attempting WebSocket subscription
VES events sent successfully
```

Phase 13 — Build minimal OCUDU PM exporter

This is the missing piece for your rApp training.

13.1 First exporter goal

Do not try to create full 3GPP PM XML immediately.

First collect raw PM:

```
OCUDU WebSocket JSON
↓
save as JSONL / CSV
↓
write to Kafka or InfluxDB
↓
train rApp
```

Create directory:

```
mkdir -p ~/ocudu-pm-exporter
cd ~/ocudu-pm-exporter
python3 -m venv .venv
source .venv/bin/activate
pip install websockets kafka-python requests pandas
```

Create collector.py:

```
import asyncio
import json
import time
from pathlib import Path

import websockets

WS_URL = "ws://<OCUDU_GNB_IP>:8001"
OUT = Path("/tmp/ocudu_pm_raw.jsonl")

async def main():
    while True:
```

```

try:
    async with websockets.connect(WS_URL) as ws:
        print(f"Connected to {WS_URL}")
        while True:
            msg = await ws.recv()
            now = time.time()
            record = {
                "timestamp_unix": now,
                "source": "ocudu-gnb",
                "raw": json.loads(msg) if msg.strip().startswith("{") else msg,
            }
            with OUT.open("a") as f:
                f.write(json.dumps(record) + "\n")
            print(record)
except Exception as e:
    print("WebSocket error:", e)
    await asyncio.sleep(5)

if __name__ == "__main__":
    asyncio.run(main())

```

Run:

```

source .venv/bin/activate
python3 collector.py

```

Verify file:

```
tail -f /tmp/ocudu_pm_raw.jsonl
```

If this works, you have raw OCUDU PM data.

13.2 Add Kafka output

Once raw collection works, send to Kafka:

```
pip install kafka-python
```

Example `collector_kafka.py`:

```

import asyncio
import json
import time

import websockets
from kafka import KafkaProducer

WS_URL = "ws://<OCUDU_GNB_IP>:8001"

```

```

KAFKA_BOOTSTRAP = "<KAFKA_HOST>:9092"
TOPIC = "ocudu.pm.raw"

producer = KafkaProducer(
    bootstrap_servers=KAFKA_BOOTSTRAP,
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
)

async def main():
    while True:
        try:
            async with websockets.connect(Ws_URL) as ws:
                print(f"Connected to {Ws_URL}")
                while True:
                    msg = await ws.recv()
                    record = {
                        "timestamp_unix": time.time(),
                        "sourceName": "ocudu-gnb",
                        "raw": json.loads(msg) if msg.strip().startswith("{") else msg,
                    }
                    producer.send(TOPIC, record)
                    producer.flush()
                    print(record)
        except Exception as e:
            print("Error:", e)
            await asyncio.sleep(5)

if __name__ == "__main__":
    asyncio.run(main())

```

Run:

```
python3 collector_kafka.py
```

Verify Kafka topic from a Kafka client pod/container:

```
kubectl get pods -n nonrtric | grep -i kafka
```

Then use your available Kafka client. Example:

```
kubectl exec -it -n nonrtric <kafka-client-pod> -- bash
```

Inside:

```

kafka-console-consumer.sh \
    --bootstrap-server <kafka-bootstrap>:9092 \
    --topic ocudu.pm.raw \
    --from-beginning

```

Phase 14 — Convert OCUDU PM to rApp training format

Your rApp training table should look like this:

```
timestamp,sourceName,node_id,cell_id,active_ues,dl_bitrate,ul_bitrate,dl_prb_usage,ul_prb_us
```

Start with simple fields:

```
timestamp
sourceName
cell_id
active_ues
dl_throughput
ul_throughput
cpu
memory
```

Then add more fields after you inspect the actual OCUDU WebSocket JSON.

Use:

```
tail -n 5 /tmp/ocudu_pm_raw.jsonl | jq .
```

Then decide field mapping.

Example normalizer:

```
def normalize(record):
    raw = record["raw"]

    return {
        "timestamp_unix": record["timestamp_unix"],
        "sourceName": record["sourceName"],
        "active_ues": raw.get("active_ues"),
        "dl_bitrate": raw.get("dl_bitrate"),
        "ul_bitrate": raw.get("ul_bitrate"),
        "cpu_usage": raw.get("cpu_usage"),
        "mem_usage": raw.get("mem_usage"),
    }
```

Do not assume field names until you inspect the real WebSocket output.

Phase 15 — Standard O1 PM fileReady path

After raw PM works, implement the standard path.

15.1 Required standard O-RAN PM objects

You need:

1. PM XML file in 3GPP TS 32.435 style
2. HTTP/HTTPS/SFTP server hosting the file
3. VES fileReady event pointing to that file
4. VES Collector reachable from exporter
5. DFC able to download the file
6. PM File Converter running
7. PM Producer running
8. rApp or Influx Logger subscription

O-RAN-SC says a PM report is an XML file with aggregated measurements over a time interval, and rApps can subscribe for chosen measurement types from measured network resources. (docs.o-ran-sc.org)

15.2 PM file-ready flow to test

Your exporter should do this every 15 seconds, 60 seconds, or 15 minutes:

```
Collect OCUDU WebSocket metrics
Aggregate over reporting period
Write PM XML file
Compress file, optional gzip
Serve file over HTTP/HTTPS
Send VES fileReady event to VES Collector
DFC downloads PM XML
PM pipeline converts and stores/distributes
```

The O-RAN-SC integration flow is: O1 component sends VES fileReady notification, VES collector forwards it to message router, file collector subscribes, downloads the file, and then the PM mapper/converter handles it. (lf-o-ran-sc.atlassian.net)

15.3 Minimal PM XML strategy

For first test, create fake but structured PM XML from OCUDU metrics:

```
ManagedElement=ocudu-gnb-1
GNBDUFunction=1
NRCellIDU=1
measTypes:
  activeUes
  dlThroughput
  ulThroughput
```



```
cpuUsage
memoryUsage
```

Your generated PM XML file name can follow:

A20260508.1200-1215_ocudu-gnb-1.xml

Host it:

```
mkdir -p ~/pmfiles
cd ~/pmfiles
python3 -m http.server 9000
```

Verify from DFC pod:

```
kubectl exec -it -n nonrtric <dfc-pod> -- sh
wget -O - http://<EXPORTER_HOST_IP>:9000/A20260508.1200-1215_ocudu-gnb-1.xml
```

15.4 Send VES fileReady event

The Data File Collector expects a fileReady VES event with fields such as:

```
{
  "event": {
    "commonEventHeader": {
      "eventName": "Noti_ocudu-gnb_FileReady",
      "sourceName": "ocudu-gnb-1",
      "changeIdentifier": "PM_MEAS_FILES"
    },
    "notificationFields": {
      "changeType": "FileReady",
      "changeIdentifier": "PM_MEAS_FILES",
      "arrayOfNamedHashMap": [
        {
          "name": "A20260508.1200-1215_ocudu-gnb-1.xml",
          "hashMap": {
            "fileFormatType": "org.3GPP.32.435#measCollec",
            "location": "http://<EXPORTER_HOST_IP>:9000/",
            "fileFormatVersion": "V10",
            "compression": "none"
          }
        }
      ]
    }
  }
}
```

The DFC documentation shows exactly this style of fileReady event: a `notificationFields` section with `changeType: FileReady`, file name, `fileFormatType`, location, version, and compression. (docs.o-ran-sc.org)

Send it to VES Collector:

```
curl -v -X POST \
  http://<VES_HOST_OR_NODE_IP>:31760/eventListener/v7 \
  -H "Content-Type: application/json" \
  -d @fileReady.json
```

If your VES endpoint uses another path/version, check your VES Collector service logs/config. Common paths vary by ONAP/O-RAN-SC release.

Phase 16 — Verify PM data reaches rApp

16.1 Check VES collector

```
kubectl logs -n nonrtric <ves-collector-pod> --tail=100 -f
```

Expected:

```
Received fileReady event
Published to Kafka/message router
```

16.2 Check DFC

```
kubectl logs -n nonrtric <dfc-pod> --tail=100 -f
```

Expected:

```
Received File Ready event
Downloading file
Stored file
Published File Publish message
```

DFC's job is to receive File Ready VES events, fetch files, store them, and publish a File Publish message for further processing. (docs.o-ran-sc.org)

16.3 Check MinIO

Open MinIO UI if using node port:

```
http://<K8S_NODE_IP>:31768
```

O-RAN-SC's RANPM deployment example lists MinIO web on node port 31768, user `admin`, password `adminadmin`. (lf-o-ran-sc.atlassian.net)

Expected:

```
PM XML file stored
Converted JSON file stored later
```

16.4 Check PM Producer

```
kubectl logs -n nonrtric <pm-producer-pod> --tail=100 -f
```

Expected:

```
New PM report available
PM report loaded
Subscription filter applied
Data sent to output Kafka topic
```

PM Producer processes PM reports and distributes requested information to subscribers over Kafka according to subscription parameters. (docs.o-ran-sc.org)

16.5 Check PM rApp

```
kubectl logs -n nonrtric <pm-rapp-pod> --tail=100 -f
```

Expected:

```
PM data received
Measurement values printed
```

If using Influx Logger:

```
kubectl logs -n nonrtric <pmlog-pod> --tail=100 -f
```

Influx Logger receives PM measurement reports from Kafka and stores them in InfluxDB. (docs.o-ran-sc.org)

Phase 17 — Recommended order for you

Do this exact order:

1. Verify Open5GS alone.
2. Verify B210 detection.
3. Verify OCUDU gNB connects to Open5GS.
4. Verify UE registration if you have UE/test SIM.

5. Verify Near-RT RIC E2 separately.
 6. Verify SMO/SDN-R UI.
 7. Verify VES Collector.
 8. Verify Non-RT RIC RANPM pods.
 9. Install OCUDU NETCONF.
 10. Verify NETCONF with netopeer2-cli.
 11. Install OCUDU O1 adapter.
 12. Verify /config-healthy.
 13. Mount OCUDU NETCONF node in SMO/SDN-R.
 14. Enable O1 adapter WebSocket + VES parameters.
 15. Collect OCUDU WebSocket PM JSON.
 16. Send raw PM to Kafka/InfluxDB.
 17. Convert PM to rApp training CSV.
 18. Later implement standard 3GPP PM XML + VES fileReady.
 19. Verify DFC → MinIO → Converter → PM Producer → rApp.
-

What you should not do yet

Do not install ZMQ now if your current goal is B210/O1/PM verification.

Do not enable:

`--ru_forward`

That is for forwarding DU NETCONF configuration to a real O-RU NETCONF server. It is not needed for B210 and not needed for ZMQ. OCUDU documents `--ru_forward` as requiring both a DU NETCONF server and an RU NETCONF server. (ocudu-docs-604e90.gitlab.io)

Do not expect B210 to provide O-RU PM or O-RU M-plane data. B210 is an SDR, not an O-RAN O-RU.

Final recommendation

For your current setup, the most realistic plan is:

Immediate:

B210 + OCUDU + Open5GS + OCUDU NETCONF + O1 Adapter + VES

For PM:

OCUDU WebSocket metrics → custom PM exporter → Kafka/InfluxDB → rApp

For standard O1 PM:

custom exporter generates 3GPP PM XML + VES fileReady → RANPM pipeline → rApp

The key truth is:

OCUDU 01 adapter alone = CM/FM + WebSocket PM support

Full 01 PM for rApp = you must add PM collector/exporter or RANPM fileReady pipeline